

Programming Basics 1



05: Arrays, Aufzählungen und Listen

Sommersemester 2025

Simon Dewert

HAW Hamburg

Fakultät Design, Medien und Information

E52, Finkenau 35, 22081 Hamburg

+49(0)40 – 428 125 714

simon.dewert@haw-hamburg.de

Übersicht dieser Lehreinheit

1. Prinzip von Arrays
2. Verwendung von Arrays
3. Mehrdimensionale Arrays
4. Aufzählungen
5. Listen - ArrayList

Prinzip von Arrays

Arrays sind **Datenfelder**, mit denen eine **festе Anzahl** zusammengehöriger Variablen zusammengefasst werden können:

Eckige Klammern signalisieren, dass es sich um ein Array handelt

datentyp `[]` arrayBezeichner = ...

Unterschiedliche Erzeugungs-Arten: siehe nächste Folie

Der lesende und schreibende Zugriff auf die Array-Elemente erfolgt über den Array-Bezeichner unter Angabe des **Index***:

Index in eckigen Klammern zum Zugriff auf ein bestimmtes Element

```
... = arrayBezeichner[index];    // lesend  
arrayBezeichner[index] = ...    // schreibend
```

* mit **0** beginnend

Erzeugung von Arrays

Arrays können grundsätzlich auf zwei unterschiedliche Arten erzeugt und initialisiert werden:

```
int[] zahlen = {2, 5, 7};           // Initialisierungsliste
```

[0]	[1]	[2]
2	5	7

```
int[] nummern = new int[3];         // Default-Werte
```

[0]	[1]	[2]
0	0	0

```
// Werte-Zuweisung  
zahlen[1] = nummern[2];
```

```
// Objekt-Zuweisung  
zahlen = nummern;
```

➡ Arrays sind **Objekte!**

Wiederholung: **Default-Werte der versch. Datentypen**

boolean	false
byte/short/int/long	0
float/double	0.0
String/Objekte	null

Iteration über Arrays

Mit einer **For-Schleife** kann man über das Array-Property **length** bequem über alle Elemente **schreibend und lesend** iterieren:

```
int[] zahlen = new int[10];  
for (int i = 0; i < zahlen.length; i++) {  
    zahlen[i] = i + 1;           // schreibend  
    System.out.println(zahlen[i]); // lesend  
}
```

Alternativ kann mit der kompakteren **For-each-Schleife** ebenfalls über alle Elemente iteriert werden, allerdings nur lesend:

Lokale Variable zum Zugriff auf die Array-Elemente

```
for (int zahl: zahlen) {  
    System.out.println(zahl); // lesend  
}
```

Aufgabe 5.1



Legen Sie zunächst ein neues Java-Projekt namens **vorlesung_05** an!
Schreiben Sie dann ein Programm **Mittelwert**, das den Mittelwert von einer beliebigen Anzahl einzugebender Messwerte (Gleitkommazahlen) auf zwei Nachkommastellen genau ausgibt!

Beispiel:

Anzahl der Messwerte: 4

- 1. Messwert: 2,5
- 2. Messwert: 3,26
- 3. Messwert: 12,801
- 4. Messwert: 6,3

Eingabe



Mittelwert: 6,22

Ausgabe

Mehrdimensionale Arrays

Mehrdimensionale Arrays sind Arrays, die wiederum Arrays als Elemente enthalten. Dadurch können **zwei-**, **drei-** oder auch **n-dimensionale Datenstrukturen** abgebildet werden:

```
datatype[][] zweidimensionalerArray = ...
```

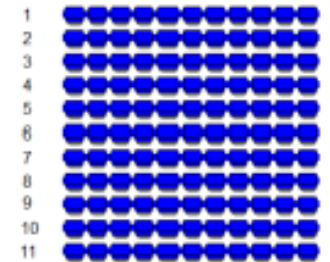
```
datatype[][][] dreidimensionalerArray = ...
```

```
datatype[]...[] n-dimensionalerArray = ...
```

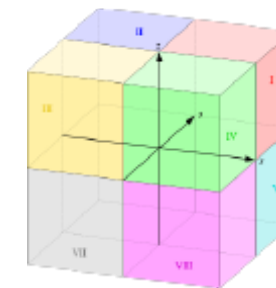
Matrix

a11	a12	a13
a21	a22	a23
a31	a32	a33

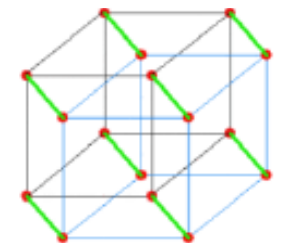
Kino-Sitzplan



2x2x2 Würfel



4D Hyper-Würfel



Erzeugung mehrdimensionaler Arrays

Die Initialisierung mehrdimensionaler Arrays geschieht analog zur Initialisierung eindimensionaler Arrays unter Beachtung der **Strukturtiefe**.

Beispiel Zweidimensionaler Array:

```
// Initialisierungsliste  
int[][] matrix = { {1,2,3,4}, {5,6,7,8} };
```

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8

```
// Default-Werte  
int[][] matrixZeros = new int[2][4];
```

	[0]	[1]	[2]	[3]
[0]	0	0	0	0
[1]	0	0	0	0

```
matrix[0][1] = matrixZeros[1][2]; // Werte-Zuweisung  
matrix = matrixZeros;           // Objekt-Zuweisung
```

Aufgabe 5.2



Schreiben Sie ein Programm **Sitzplan**, bei dem man einen bestimmten Sitz in einem Kino mit **5 Reihen** und **10 Sitzen pro Reihe** belegen kann!

Der Benutzer soll hierzu gefragt werden, in welcher Reihe welcher Sitz belegt werden soll. Diese Information soll in einem zweidimensionalen Array gespeichert werden, der anschließend "**grafisch**" dargestellt wird.

Beispiel:

```
Reihe (1-5) : 3  
Sitz: 7
```

Eingabe

```
0000000000  
0000000000  
0000001000  
0000000000  
0000000000
```

Ausgabe

Aufzählungen - Enums

Aufzählungen (engl. "Enumerations") sind **komplexe Datentypen**, die durch explizite **Aufzählung ihrer Werte** definiert werden.

Die Werte werden **außerhalb von Methoden** und üblicherweise – ähnlich wie Konstanten – in Großbuchstaben formuliert:

```
public class MeineKlasse
{
    enum MeineAufzaehlung {WERT1, WERT2, WERT3, ... }
    [...]
}
```

Großer Anfangsbuchstabe

in Großbuchstaben

Typische Anwendungsgebiete sind z. B. Wochentage, Farben etc., um die **Verständlichkeit** und **Sicherheit** des Codes zu erhöhen.

Verwendung von Aufzählungen

Wie bei anderen Datentypen auch können für Aufzählungen **Variablen** angelegt und mit Werten versehen werden, die zum Beispiel dann in einer Switch-Verzweigung **geprüft** werden:

```
enum Zustand {ZUSTAND1, ZUSTAND2, [...]}    Definition nicht innerhalb der Methode!

public static void main(String[] args)
{
    Zustand meinZustand = Zustand.ZUSTAND1;    // Zustand zuweisen

    switch (meinZustand)                       // Zustand abfragen
    {
        case ZUSTAND1:
            [...]

        case ZUSTAND2:
            [...]
    }
}
```

Aufgabe 5.3



Schreiben Sie ein Programm **Wuerfel**, das einen **Würfel mit sechs Seiten** simuliert! Verwenden Sie zur Abbildung der Würfelseiten eine Aufzählung!

Hinweis: Mit der (statischen) Methode `values()` einer Aufzählung erhalten Sie einen **Array all ihrer möglichen Werte**.

Beispiel:

Der Würfel zeigt VIER

Ausgabe

Nochmal würfeln (j/n)? j

Eingabe

Der Würfel zeigt ZWEI

Ausgabe

Nochmal würfeln (j/n)? n

Eingabe

Listen - ArrayList

Listen bieten einen komfortableren Umgang als Arrays und sind dabei flexibel in der Anzahl der Elemente.

Es gibt viele verschiedene Arten von Listen ➡

Links zur Dokumentation der Listen ([AbstractList](#), [AbstractSequentialList](#), [ArrayList](#), [AttributeList](#), [CopyOnWriteArrayList](#), [LinkedList](#), [RoleList](#), [RoleUnresolvedList](#), [Stack](#), [Vector](#))

Wir wollen uns erstmal nur mit der einfachen ArrayList befassen:

```
import java.util.List;
import java.util.ArrayList;
[...]
List<String> ListenName = new ArrayList<String>();
```





ArrayList

Neue Liste erzeugen:

```
List<int> ListenName = new ArrayList<String>();
```

Hinzufügen von Elementen:

```
ListenName.add("Hallo");
```

Auslesen erfolgt wie bei normalen Arrays über den Index:

```
System.out.println(ListenName.get(0)); // Ausgabe:  
Hallo
```

Anders als beim Array (`.length`) wird über `.size()` die Länge (Anzahl der Elemente) bestimmt.

Achtung: hier muss der **selbe** Datentyp stehen!
Außerdem sind **keine** **Primitiven** Datentypen zulässig.
Nutzt die **Wrapper!**

Aufgabe 5.4



Wie Aufgabe 5.1: Schreiben Sie ein Programm **MittelwertMitArrayList**, das den Mittelwert von einer beliebigen Anzahl einzugebender Messwerte (Gleitkommazahlen) ausgibt! Verwenden Sie jetzt eine **ArrayListe** statt des Arrays!

Nach jeder Eingabe soll die neue Zahl der Liste hinzugefügt werden und der Mittelwert der gesamten Liste ausgegeben werden:

Beispiel:

1. Element:	1,2	Eingabe
1.2 - Mittelwert:	1,2	Ausgabe
2. Element:	2,4	Eingabe
1.2 2.4 - Mittelwert:	1,8	Ausgabe

Bonus Aufgabe



Schreiben Sie ein Java-Programm, das eine Liste von Namen verwaltet.
Verwenden Sie dafür eine `ArrayList<String>`.

Es soll über ein Konsolenmenü folgende Funktionen anbieten:

Hinweis:

- Verwenden Sie die Klasse `Scanner` zur Eingabe
- Nutzen Sie die Methoden von `ArrayList`, wie:
`add()`, `remove()`, `contains()`, `sort()`, `get()`, `size()`, etc.
- Alle Namen sind vom Typ `String`

```
== Namensliste ==  
1. Namen hinzufügen  
2. Namen entfernen  
3. Namen suchen  
4. Namen sortieren  
5. Alle Namen anzeigen  
6. Beenden  
Ihre Wahl:
```

Viele Dank für Ihre Aufmerksamkeit



Hausaufgabe



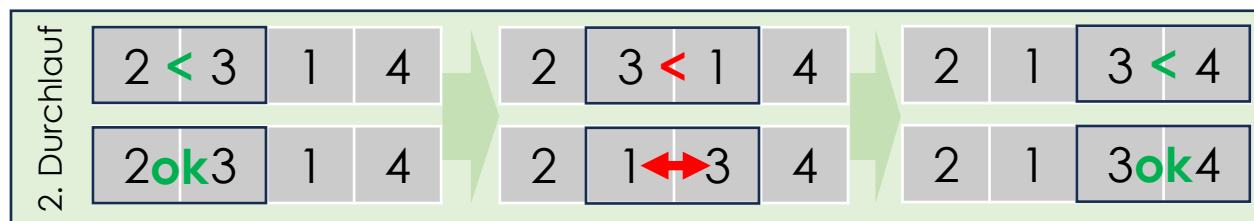
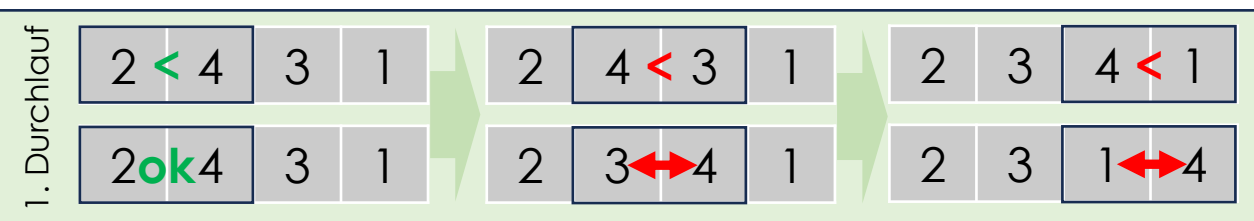
Schreiben Sie ein Programm namens **ExtremwerteSortierung**, dass die Extremwerte ermittelt, indem es die beliebigen Zahlen eines Arrays der Größe nach sortiert!

Anleitung: Iterieren Sie über den Array und vertauschen Sie zwei benachbarte Werte, wenn der "linke" Wert größer ist als der "rechte". Iterieren Sie so oft über den Array, bis keine Vertauschung mehr stattfindet!

Startwerte (Beispiel): 4 2 3 1

Automatische Sortierung wie `Arrays.sort()` werden als nicht abgegeben gewertet!

Geben Sie nach jedem Durchlauf das Array in die Konsole aus



usw... bis alles sortiert ist